

CS-250: Lab 06

AVL Trees

Muhammad Mujtaba

CMD ID: 540040

mmujtaba.bese25seecs@seecs.edu.pk

Class: BESE 16B

Batch: 2k25

AVL Tree DS Used:

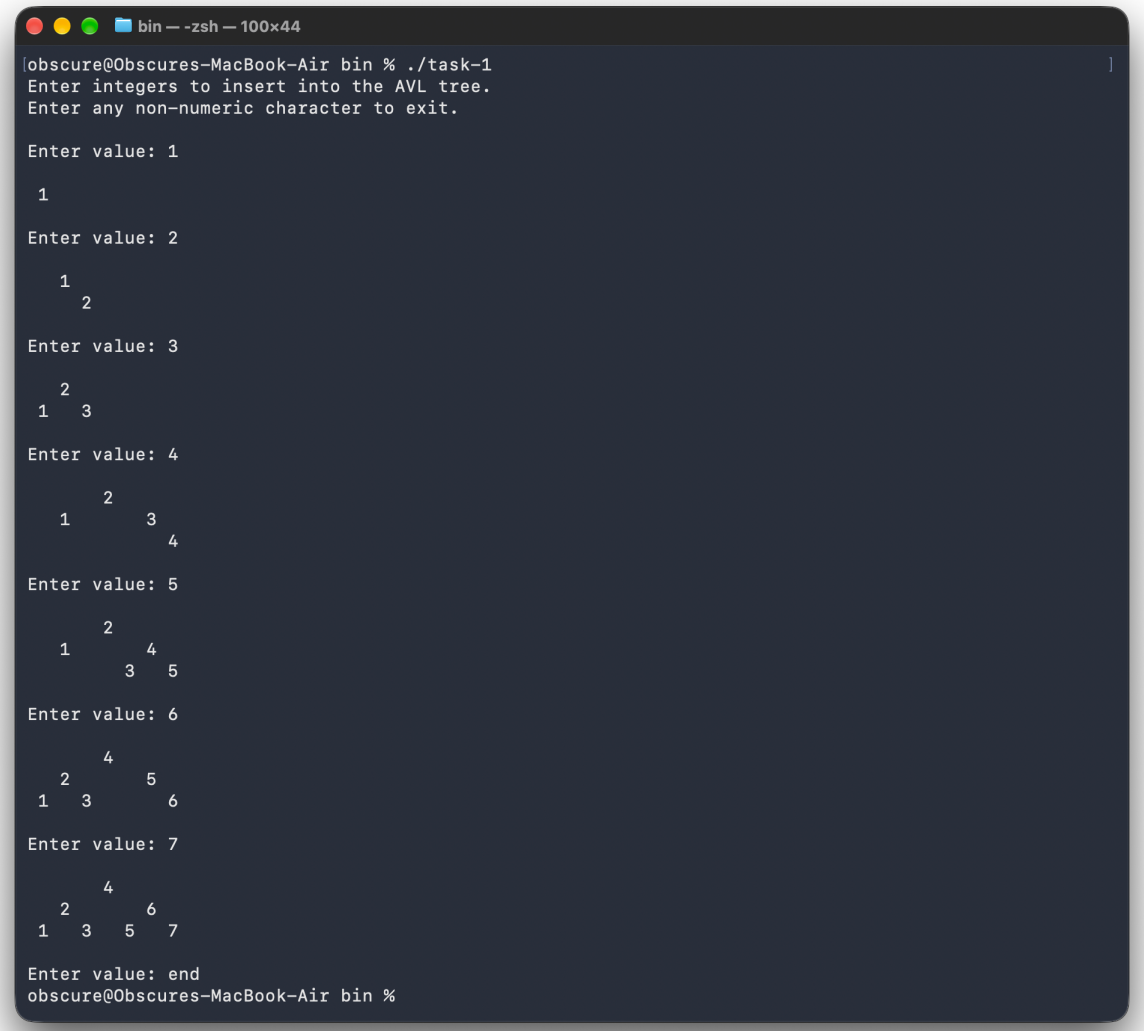
See avl_tree.hpp and avl_tree.cpp

GitHub Repo Link: https://github.com/mmujtaba25/CS-250

Zip File for only Lab 06: https://github.com/mmujtaba25/CS-250/releases/tag/lab-06

Task 1:

See avl_tree.hpp, avl_tree.cpp and task1.cpp.



Task 1 Driver (task1.cpp):

```cpp
#include "avl_tree.hpp"

#include <print>
#include <iostream>
#include <string>
#include <cstdio>

void printTree(const AVLTree &tree);
void printLevel(const AVLTreeNode *node, int level, int spacing);

int main()
{
    AVLTree tree;

    std::println("Enter integers to insert into the AVL tree.");
    std::println("Enter any non-numeric character to exit.\n");

    while (true)
    {
        int value;

        std::print("Enter value: ");
        std::fflush(stdout);

        if (!(std::cin >> value))
            break;

        tree.insert(value);

        std::println();
        printTree(tree);

        std::println();
    }

    return 0;
}

// MARK: Helpers

void printLevel(const AVLTreeNode *node, int level, int spacing)
{
    if (level == 0)
    {
        std::print("{:^{}}", node ? std::to_string(node->data) : " ", spacing);
        return;
    }

    // recursively print each levvel
    printLevel(node ? node->left : nullptr, level - 1, spacing);
    printLevel(node ? node->right : nullptr, level - 1, spacing);
}

void printTree(const AVLTree &tree)
{
    const AVLTreeNode *root = tree.root();

    if (root == nullptr)
        return;

    int height = root->height;

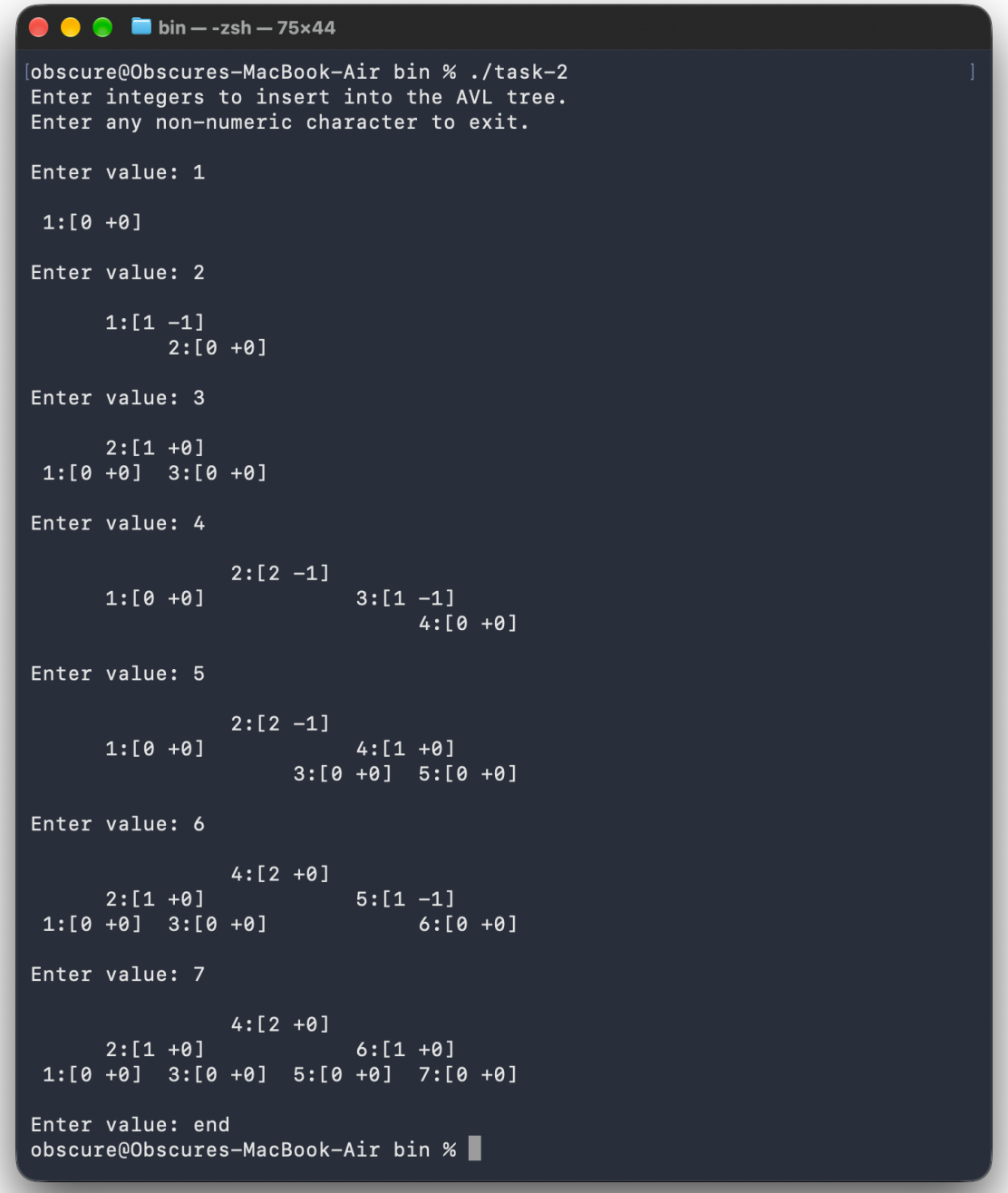
    for (int level = 0; level < height; level++)
    {
        int spacing = 4 * (1 << (height - level - 1));

        printLevel(root, level, spacing);

        std::println();
    }
}
```

Task 2:

See avl_tree.hpp, avl_tree.cpp and task2.cpp.



Task 2 Driver (task2.cpp):

```cpp
#include "avl_tree.hpp"

#include <print>
#include <format>
#include <iostream>
#include <string>
#include <algorithm>
#include <cstdio>

std::string nodeLabel(const AVLTreeNode *node);
size_t maxLabelWidth(const AVLTreeNode *node);

void printTree(const AVLTree &tree);
void printLevel(const AVLTreeNode *node, int level, int spacing);

int main()
{
    AVLTree tree;

    std::println("Enter integers to insert into the AVL tree.");
    std::println("Enter any non-numeric character to exit.\n");

    while (true)
    {
        int value;

        std::print("Enter value: ");
        std::fflush(stdout);

        if (!(std::cin >> value))
            break;

        tree.insert(value);

        std::println();
        printTree(tree);

        std::println();
    }

    return 0;
}

// MARK: Helpers

std::string nodeLabel(const AVLTreeNode *node)
{
    if (node == nullptr)
        return "";

    int leftHeight = node->left ? node->left->height : 0;
    int rightHeight = node->right ? node->right->height : 0;

    int balance = leftHeight - rightHeight;

    return std::format("{}:[{} {:+}]", node->data, node->height - 1, balance);
}

size_t maxLabelWidth(const AVLTreeNode *node)
{
    if (node == nullptr)
        return 1;

    return std::max({nodeLabel(node).size(), maxLabelWidth(node->left), maxLabelWidth(node->right)});
}
```

```cpp
void printLevel(const AVLTreeNode *node, int level, int spacing)
{
    if (level == 0)
    {
        std::print("{:^{}}", nodeLabel(node), spacing);
        return;
    }

    // recursively print each levvel
    printLevel(node ? node->left : nullptr, level - 1, spacing);
    printLevel(node ? node->right : nullptr, level - 1, spacing);
}

void printTree(const AVLTree &tree)
{
    const AVLTreeNode *root = tree.root();

    if (root == nullptr)
    {
        std::println("(empty)");
        return;
    }

    int height = root->height;
    int width = static_cast<int>(maxLabelWidth(root)) + 2;

    for (int level = 0; level < height; level++)
    {
        int spacing = width * (1 << (height - level - 1));

        printLevel(root, level, spacing);

        std::println();
    }
}
```